Partial and Constrained Level Planarity*

Guido Brückner† Ignaz Rutter‡

Abstract

Let $G = (V, E)$ be a directed graph and $\ell\colon V \to [k] := \{1, \dots, k\}$ a level assignment such that $\ell(u) < \ell(v)$ for all directed edges $(u, v) \in E$. A level planar drawing of G is a drawing of G where each vertex v is mapped to a unique point on the horizontal line ℓ_i with y-coordinate $\ell(v)$, and each edge is drawn as a y-monotone curve between its endpoints such that no two curves cross in their interior.

In the problem Constrained Level Planarity (CLP for short), we are further given a partial ordering $\prec_i$ of $V_i := \ell^{-1}(i)$ for $i \in [k]$, and we seek a level planar drawing where the order of the vertices on ℓ_i is a linear extension of $\prec_i$. A special case of this is the problem Partial Level Planarity (PLP for short), where we are asked to extend a given level-planar drawing $\mathcal{H}$ of a subgraph $H \subseteq G$ to a complete drawing $\mathcal{G}$ of G without modifying the given drawing, i.e., the restriction of $\mathcal{G}$ to H must coincide with $\mathcal{H}$.

We give a simple polynomial-time algorithm with running time $\mathcal{O}(n^5)$ for CLP of single-source graphs that is based on a simplified version of an existing level-planarity testing algorithm for single-source graphs. We introduce a modified type of PQ-tree data structure that is capable of efficiently handling the arising constraints to improve the running time to $O(n + k\ell)$, where ℓ is the size of the constraints. We complement this result by showing that PLP is NP-complete even in very restricted cases. In particular, PLP remains NP-complete even when G is a subdivision of a triconnected planar graph with bounded degree.

1 Introduction

A *k-level graph* is a directed graph $G = (V, E)$ together with a leveling $\ell\colon V \to [k] := \{1, \dots, k\}$ that assigns each vertex to one of k levels so that $\ell(u) < \ell(v)$ for each edge $(u, v) \in E$. A *level drawing* is a drawing of G that maps each vertex v to a point on the horizontal line with y-coordinate $\ell(v)$ and each edge to a y-monotone curve between its endpoints. A crossing-free level drawing is *level planar*, and a graph is *level planar* if it admits a level planar drawing.

In this paper, we study a generalization of the level planarity testing problem where, in addition to the k-level graph G, we are given *constraints* on the order of the vertices in each level in the form of a partial order $\prec_i$ for each level $i \in [k]$. The task is to determine a level-planar drawing of G *compatible* with the constraints, i.e., such that the vertex order in each level i is a linear extension of $\prec_i$. We call this problem Constrained Level Planarity or CLP for short.

A strongly related problem is the problem of extending a given drawing $\mathcal{H}$ of a subgraph $H \subseteq G$, also called *partial drawing*, to a level-planar drawing $\mathcal{G}$ of G without changing the given drawing $\mathcal{H}$. That is, the restriction of $\mathcal{G}$ to H must coincide with $\mathcal{H}$. We call such a drawing $\mathcal{G}$ an *extension*. The problem of deciding whether a given partial drawing admits an extension is called Partial Level Planarity, or PLP for short. Of course, taking for each level i the order $\prec_i$ as the vertex order on level i in $\mathcal{H}$ yields a set of constraints. An extension necessarily is compatible with all these constraints. Later, we will see that for so-called proper graphs these instances are actually equivalent. Since both problems can be reduced to the corresponding problem on proper graphs it turns out that indeed PLP can be seen as a special case of CLP.

Related Work. Historically, level drawings were among the first drawing styles that were studied systematically with the introduction of the famous Sugiyama framework [35]. Level drawings are frequently used for visualizing hierarchical data and there is extensive research on every step of the framework; see [19] for a survey. Due to the importance of crossings on the visual clarity of drawings, the concept of level planarity has also received considerable attention. Di Battista and Nardelli [14] gave the first efficient recognition algorithm for the subclass of single-source graphs. Jünger and Leipert [25] solved the general case and gave a linear-time algorithm for testing whether a given level graph is level planar. Later they improved their algorithm to also output a corresponding level planar embedding, if it exists, in the same running time [24]. Their algorithm is, however, quite complicated. As a result, Randerath

*Work partially supported by grant WA 654/21-1 of the German Research Foundation (DFG)

†Karlsruhe Institute of Technology, Karlsruhe, Germany, brueckner@kit.edu

‡TU Eindhoven, Eindhoven, The Netherlands, i.rutter@tue.nl

et al. [32] gave a much simpler recognition algorithm with running time $\mathcal{O}(n^2)$, and Harrigan and Healy [18] formulated a quadratic-time recognition and embedding algorithm. There is also a quadratic-time algorithm based on a Hanani-Tutte characterization [16]. Moreover, radial variants, where the levels are represented by concentric circles rather than horizontal lines [5], as well as variants on cylinders [11] and on the torus [4] have been considered.

Constrained versions of graph representations and planarity variants are another type widely studied problem as, for visualization purposes, it is often desirable to find visualizations that satisfy additional properties. Some constrained versions of level planarity have been considered. Harrigan and Healy [18] and Angelini et al. [2] both study level planarity where vertices of the same level must be ordered consistently with different kinds of *constraint trees*. Sources of such constraints include restrictions imposed by a user, e.g., by specifying the left-to-right order of some vertices or even fixing a part of the drawing, or as a consistency requirement stemming from the dynamic nature of a network, where one considers the restrictions imposed by stable parts of a network as constraints on the representation of the next snapshot. The latter leads to a *partial drawing extension* problem. More formally, a *partially drawn graph* is a triplet $(G, H, \mathcal{H})$, where G is a graph, $H \subseteq G$ is a subgraph and $\mathcal{H}$ is a drawing of H. The partial drawing extension problem with input $(G, H, \mathcal{H})$ asks to complete the drawing $\mathcal{H}$ to a drawing $\mathcal{G}$ of G without modifying the given subdrawing of $\mathcal{H}$, i.e., the restriction of $\mathcal{G}$ to H is identical to $\mathcal{H}$. The partial drawing extension problem, and the related simultaneous drawing problem have received considerable attention in the last years.

Angelini et al. [3] give a linear-time algorithm for partial drawing extension of *topological* planar drawings, where edges are represented by arbitrary, non-crossing curves between their endpoints. In this case, the positive instances have also been characterized via forbidden subgraphs [23] and there exists a Hanani-Tutte characterization [33], which also leads to a polynomial-time algorithm. In contrast, for planar straight-line drawings, the partial drawing extension problem is NP-complete [31], though it becomes polynomial-time solvable if H is biconnected and $\mathcal{H}$ is a convex drawing [30]. Moreover, the problem has also been studied under the name partial representation extension for graph representations that are not node-link representations, in particular for different types of intersection representations, namely for interval representations [29, 9], for proper and unit interval representations [27], for chordal graphs (which are intersection graphs of subtrees of a tree) [28], for permutation and function graphs [26] and, recently, for circle graphs [12].

We point out that the partial drawing problem is strongly related to the *simultaneous drawing problem*, whose input consists of two graphs G_1 and G_2 sharing a subgraph $G = G_1 \cap G_2$. The problem is to decide whether there exist drawings of G_1 and G_2 that coincide on G. This is equivalent to finding a drawing of G that extends to drawings of G_1 and G_2. It is known that the problem is NP-complete for planar straight-line drawings [15], though some special cases have recently been shown to be polynomial-time solvable[17]. For topological planar drawings, this problem is called SEFE and despite significant progress in the last years [1, 8, 6, 9], the complexity status is still open; see [7] for a survey. The simultaneous representation problem for intersection representations was introduced by Jampani and Lubiw, who studied permutation and chordal graphs [22] and interval graphs [21]. Bläsius and Rutter [9] improved the running time for simultaneous interval representations to linear. Recently, level planarity has been considered in this setting, too; simultaneous level planarity is NP-complete for two graphs on three levels, as well as for three graphs on two levels, an efficient algorithm exists only for two graphs on two levels [4].

Contribution and Outline. We study the level planarity problem subject to constraints and in the context of partial drawings. We present preliminaries in Section 2. In particular, we show that, indeed, PLP reduces to CLP. In Section 3 we present a simplified version of the algorithm of Di Battista and Nardelli [14] for the single-source case. We then introduce the order graph, a data structure that allows us to efficiently intersect an implicit representation of the set of all level-planar drawings of a graph with an implicit representation of a set of (not necessarily planar) level drawings that satisfy the constraints. Since the latter set is guaranteed to contain all level-planar drawings that satisfy the constraints, this allows us to solve CLP for single-source graphs in time $\mathcal{O}(n^5 + \ell)$, where ℓ is the size of the constraints. We then refine the algorithm and develop a modified version of a PQ-tree, which we call *constrained PQ-tree*, which handles both the planarity and the constraints in the same data structure. With this modification the algorithm runs in time $\mathcal{O}(n + k\ell)$ for proper k-level graphs. In Section 4 we complement these results by showing that PLP is NP-complete even in quite restricted cases, in particular, even when it is a subdivision of a 3-connected planar graph (i.e., its combinatorial embedding is essentially fixed), the graph has fixed maximum degree and the graph has only a constant number of sources per level. This shows that our algorithmic results are likely tight.

Moreover, while a Hanani-Tutte-style characterization exists for level planarity, our hardness result rules out the existence of such a characterization for partial level planarity. This contrasts the situation for usual (topological) planarity, where Hanani-Tutte characterizations exist for both variants [33].

2 Preliminaries

A k-level graph is *proper* if and only if $\ell(v) = \ell(u) + 1$ for each $(u, v) \in E$. Every k-level graph $G = (V, E)$ can be transformed into a proper graph $G' = (V', E')$ by subdividing edges. Likewise, a (not necessarily planar) level drawing Γ of G can be transformed into a level drawing Γ' of G'. Thus, though in the following we restrict our treatment to proper level graphs, this is not a loss of generality, though the resulting proper graph my have size $\Theta(n^2)$.

Combinatorially, a level drawing Γ of a proper k-level graph $G = (V, E)$ can be described by the linear orderings $\prec_i$ of the vertices in V_i along the horizontal line ℓ_i with y-coordinate i. By defining $u \prec v$ if $\ell(u) < \ell(v)$ or if $\ell(u) = \ell(v)$ and $u \prec_i v$, we can describe a level drawing by a single linear ordering $\prec$ of V, which lists the vertices in $V_1, \ldots, V_k$ in their left-to-right orders. Di Battista and Nardelli give the following characterization of planar level drawings.

LEMMA 2.1. ([14, LEMMA 1]) *Let $G = (V, E)$ be a proper k-level graph and let $\prec$ be a drawing of G. Then $\prec$ is planar if and only if for any distinct vertices $u, w \in V_j$ and $v, x \in V_{j+1}$, $j \in [k]$ with $(u, v), (w, x) \in E$ it is $u \prec w$ if and only if $v \prec x$.*

Two vertex pairs ss' with $s, s' \in V_i$ and tt' with $t, t' \in V_j$ are *equivalent*, written $ss' \equiv tt'$ if in every level-planar drawing $\prec$ of G it is either $s \prec s'$ and $t \prec t'$ or $s' \prec s$ and $t' \prec t$. Lemma 2.1 allows us to determine a first set of equivalent vertex pairs.

Of course, if $ss' \equiv tt'$ and $tt' \equiv uu'$, transitivity implies $ss' \equiv uu'$. More generally, we can obtain additional equivalent pairs by considering suitable paths in G as follows. Let $\text{lace}(a, b)$ denote the set of all paths whose first vertex is in level a, whose last vertex is in level b, and whose interior vertices v satisfy $\ell(v) \in [a, b]$. We denote a path with endpoints s and t by $\langle s, t \rangle$.

LEMMA 2.2. ([14, LEMMA 2]) *Let G be a proper k-level graph and let $\langle s, t \rangle$, $\langle s', t' \rangle \in \text{lace}(a, b)$ with $1 \leq a < b \leq k$ be two vertex-disjoint paths. Then $ss' \equiv tt'$.*

Drawing constraints for a proper level graph $G = (V, E)$ are expressed as a partial ordering $\prec'$ of V. A drawing $\prec$ of G satisfies the constraints $\prec'$ if and only if for every $u, v \in V$ with $u \prec' v$ it is also $u \prec v$. The size ℓ of $\prec'$ is the number of distinct vertex pairs that are related by $\prec'$. In general, ℓ is quadratic in $n = |V|$.

If $\prec'$ is a total order of a subset $V' \subseteq V$, it exhaustively describes the drawing of the graph $G' = G[V']$. Hence, such a $\prec'$ is called a *partial drawing* of G. Note that if a partial drawing is stored transitively reduced, its size is linear in n. Of course, partial drawings are a subset of drawing constraints. Moreover, in a straight-line level planar drawing of a proper graph, the vertices can be moved horizontally without creating crossings as long as their order within the levels does not change. (Lemma 2.1). Thus, if a drawing satisfying the order constraints of an instance of PLP exists, the vertices of the drawing can always be moved so that the partial drawing is preserved. It follows that PLP is indeed a special case of CLP.

LEMMA 2.3. *For proper level graphs,* PLP *reduces to* CLP *in linear time.*

Conversely, drawing constraints are strictly more powerful than partial drawings. For example, with drawing constraints it is possible to express the constraints $u \prec' v, u \prec' w$ without deciding whether $v \prec' w$ or $w \prec' v$. This is not possible with partial drawings, because the totality property requires that v and w be related by the partial drawing, too.

PQ-trees. Booth and Lueker [10] introduced the PQ-tree data structure, which is capable of representing sets of linear orders on a ground set. This data structure was later generalized to PC-trees by Hsu and Shih [34], which represent circular orders of a ground set. Hsu and McConnell later presented a considerable simplification of PC-trees [20].

A PC-tree T is an unrooted, ordered tree where the inner nodes are either P-nodes or C-nodes, and the leaves are in one-to-one correspondence with the elements of a finite ground set U, called the *yield* of T, i.e., $U = \text{yield}(T)$. Any P-node has at least three neighbors, and any C-node has at least four neighbors, so the size of T is linear in $|U|$. A traversal of the leaves of T leads to a cyclic order of the elements in U, called the *frontier of* T and denoted by $\text{frontier}(T)$. Even though PC-trees are unrooted, an internal rooting at a freely choosable leaf is maintained for the implementation, which allows for the notion of *parents* and *children*.

Shih and Hsu [34] describe two equivalence transformations for PC-trees. The order of the neighbors of a P-node may be arbitrarily rearranged, whereas the order of the neighbors of a C-node may be reversed. Performing such an equivalence transformation on a PC-tree T results in a new PC-tree T', which generally has

a different frontier. Two PC-trees T and T' are called *equivalent*, i.e., $T \equiv T'$, if and only if T' can be obtained from T by a number of equivalence transformations. The set of *consistent* permutations represented by T is consistent$(T) = \{\text{frontier}(T') \mid T' \equiv T\}$.

The usefulness of PC-trees arises from the operation update(T, S). For a PC-tree T and a subset $S \subseteq$ yield(T) of its yield, this operation returns a new PC-tree T' that limits the cyclic orders represented by T to those in which the elements in S appear consecutively. If there are no such cyclic orders, the result is the *null tree*, which formally represents the empty set of orderings of a ground set, i.e., if T' is the null tree, then consistent$(T') = \emptyset$.

The update operation consists of three steps; see Figure 1 for an illustration. The first step is to label any leaf $v \in S$ as *black* and all other nodes as *white*, and then label any inner nodes as black if all its neighbors except one are black. Edges that are incident to a black node are also called black. An edge e of T is called *terminal* if both connected trees in $T - e$ contain both black and white leaves. All other edges are white. The terminal edges form a path in T, the *terminal path*. In the second step, a new C-node x is created and the edges of the terminal path are deleted. Then, for each node y on the terminal path a split node y' is created, all black neighbors are moved from y to y', and y and y' are connected to x. Finally, in the third step, all edges from x to C-node neighbors are contracted. As a cleanup, leaves that do not correspond to an element of the ground set are removed, and internal degree-2 nodes are contracted to obtain a well-defined PC-tree.

LEMMA 2.4. ([20, LEMMA 4.7]) *The* update(T, S) *operation can be performed in* $\mathcal{O}(p + |S|)$ *time, where* p *is the length of the terminal path in* T.

A PQ-tree is essentially a rooted PC-tree where C-nodes are called Q-nodes. Since the tree is rooted, child edges of inner nodes appear in linear orders. The order of the child edges of a node may be arbitrarily rearranged for P-nodes, and reversed for Q-nodes. Shih and Hsu show that PC-trees can be used as a drop-in replacement for PQ-trees by adding a special root element to the tree and splitting the cyclic orders at that element into linear orders. This special root element can be chosen independently from the internal root mentioned above.

3 Single-Source Graphs

Di Battista and Nardelli [14] presented a linear-time algorithm solving LEVEL PLANARITY for single-source proper level graphs. In this section, we present a simplified variant of their algorithm with the same running time. We then extend this algorithm to a CLP algorithm with running time $\mathcal{O}(n^5 + \ell)$, where ℓ is the size of the constraints $\prec'$, by introducing the new concept of *order graphs*. Finally, we introduce *constrained PQ-trees*, which are modified PQ-trees that are capable of efficiently maintaining additional constraints. With the use of constrained PQ-trees, we improve the running time of our algorithm to $\mathcal{O}(n + k\ell)$.

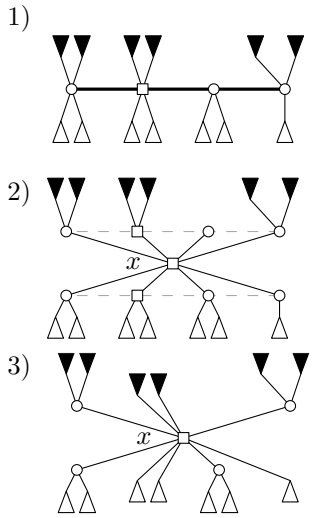

Figure 1: The three-step update procedure for PC-trees; C-nodes are represented by squares, P-nodes by disks. 1) The terminal path is bold, the subtrees consisting of only black and of only white vertices are represented by black and white triangles, respectively. 2) The terminal path is split and a new C-node x is connected to all vertices of the terminal path. 3) Contraction of edges between x and adjacent C-nodes and cleanup.

3.1 A Simple LEVEL PLANARITY Algorithm

For a proper k-level planar graph, we denote by G_j the subgraph induced by the vertices on levels $1, \ldots, j$. Moreover, by $E_j = (V_j \times V_{j+1}) \cap E$, we denote the edges of G between levels j and $j+1$. We start out by making the following observation about planar drawings of level graphs.

LEMMA 3.1. *Let* G *be a proper* k*-level graph together with a planar drawing* $\prec$. *Let* $j \in [k-1]$ *and for some* $\lambda \in (0,1)$ *let the order of edges in* E_j *in which they intersect with the horizontal line* $y = j + \lambda$ *be* $(u_1, v_1) \prec (u_2, v_2) \prec \ldots \prec (u_t, v_t)$. *Then the edge endpoints are ordered consecutively, i.e., from* $u_a = u_b$ *with* $1 \le a < b \le t$ *it follows that* $u_a = u_{a+1} = \ldots = u_b$, *and from* $v_a = v_b$ *with* $1 \le a < b \le t$ *it follows that* $v_a = v_{a+1} = \ldots = v_b$.

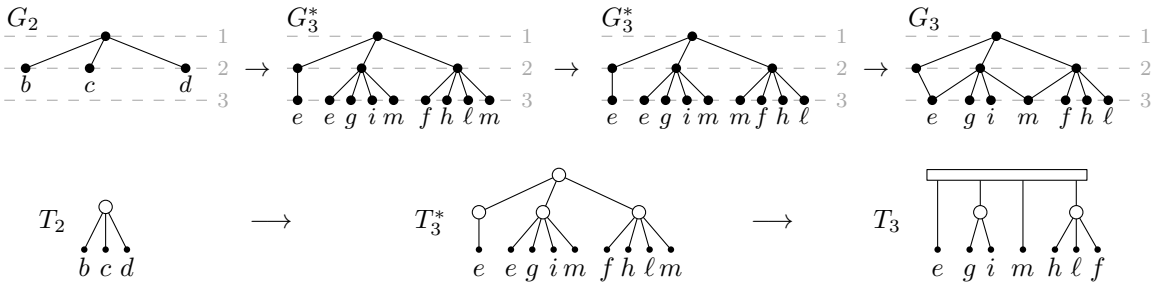

Figure 2: Illustration of Di Battista and Nardelli's algorithm applied to the graph shown on the left. Starting from the PQ-tree T_2, the middle PQ-tree is generated by adding all level-3 neighbors as children to b, c, d. Then, multiple occurrences are made consecutive and replaced by a single vertex, leading to T_3, as shown in the right.

Proof. Assume that $\prec$ is planar and that there exists $u_c \neq u_a = u_b$ with $a < c < b$. Then it is $(u_a, v_a) \prec (u_c, v_c) \prec (u_b = u_a, v_b)$. From $u_a \neq u_c \neq u_b$ it follows that either $u_a \prec u_c$ or $u_c \prec u_b = u_a$ holds true. If it is $u_a \prec u_c$, then (u_c, v_c) intersects with $(u_b = u_a, v_b)$. Conversely, if it is $u_c \prec u_a$, then (u_c, v_c) intersects with (u_a, v_a). With Lemma 2.1, this contradicts the planarity of $\prec$, so no such u_c can exist. The same idea can be used to show that there exists no $v_c \neq v_a$ with $a < c < b$. □

The idea of Di Battista and Nardelli is to perform a top-down sweep of the single-source proper k-level graph, successively visiting each level $j \in [k]$. When going from level j to level $j+1$, they compute how planar drawings of G_j can be extended to planar drawings of G_{j+1}. To efficiently represent all possible planar drawings simultaneously, they use PQ-trees.

Our algorithm follows this approach, computing the same PQ-trees, albeit in a simpler way. Consider how a planar drawing $\prec_j$ of G_j can be extended to a drawing $\prec_{j+1}$ of G_{j+1}. Start by drawing non-intersecting edge stubs protruding from u for any edge $(u, v) \in E_j$. Lemma 3.1 states that in any planar drawing, these edge stubs have to be ordered so that identical endpoints are consecutive. The stubs can then be extended to complete edges meeting at their shared endpoint without causing any edge crossings.

The algorithm is as follows. Instead of considering individual planar drawings, for each level $j \in [k]$ a PQ-tree T_j is computed that represents the orders of level-j vertices across all planar drawings of G_j; see Figure 2 for an example. The tree T_1 consists of a single leaf, the source of G. Given a tree T_j, the tree T_{j+1} for the subsequent level is generated as follows. If T_j is empty, so is T_{j+1}. If T_j is non-empty, consider each leaf u of T_j. If u has no child edges in E_j, mark u as inactive. Otherwise, add each child edge as a child of u in T_j. Once all leaves have been considered, post-order traverse through T_j and mark any node as inactive if all its children are inactive. All other nodes remain active. Edge leafs with identical endpoints v are then made consecutive in T_j using the update operation. Recalling the PC-tree update operation, the edge leafs are now consecutive black subtrees attached to a C-node. Finally, these black subtrees are replaced by a single leaf v. The following Lemma establishes the correctness of this algorithm.

LEMMA 3.2. *Let G be a single-source proper k-level graph and let T_j be the PQ-tree for some level $j \in [k]$. It is $(v_1, v_2, \ldots, v_t) \in$ consistent(T_j) if and only if there exists a planar drawing $\prec_j$ of G_j that satisfies $v_1 \prec_j v_2 \prec_j \ldots \prec_j v_t$.*

From Lemma 3.2 it follows that if T_j is empty for any $j \in [k]$, then G is not level planar. Otherwise, if T_k is non-empty, G is level planar. For a family of sets $\mathcal{S} = \{S_1, S_2, \ldots, S_t\}$ all update(T, S_i) operations are feasible in a total running time of $\mathcal{O}(|T| + |\mathcal{S}| + \sum_{i=1}^{t} |S_i|)$ [10]. It is $|T_j| \in \mathcal{O}(n_j + n_{j+1})$, $|\mathcal{S}| \in \mathcal{O}(n_{j+1})$ and since $\mathcal{S}$ is a family of disjoint sets $\sum_{i=1}^{t} |S_i| \in \mathcal{O}(n_{j+1})$. This gives linear running time for the entire algorithm. Di Battista and Nardelli also give an important link of inner nodes of the PQ-tree T_j to parts of the graph G_j.

LEMMA 3.3. ([14]) *Let G be a single-source proper level graph and let T_j be the PQ-tree for some level $j \in [k]$. Every P-node in T_j corresponds to a cutvertex in G_j, and every Q-node in T_j corresponds to a maximal biconnected component of G_j.*

3.2 A Polynomial-Time CLP Algorithm

The algorithm from the previous section uses PQ-trees T_j to restrict all drawings of a level graph to just those that are level planar, represented by consistent(T_j). This section introduces order graphs P_j, which are used in a similar way to restrict

all drawings of a level graph to a subset of the drawings compatible with the constraints $\prec'$ that contains all level-planar drawings compatible with $\prec'$, represented by consistent(P_j). Together, these two data structures restrict all drawings of a level graph to those that are planar *and* satisfy the given constraints $\prec'$, represented by consistent$(T_j, P_j) :=$ consistent$(T_j) \cap$ consistent(P_j). We show that there exists a level-planar drawing of G compatible with $\prec'$ if and only if consistent(T_j, P_j) is non-empty for all $j \in [k]$. The remainder of this section describes the order graph data structure, the generation of the order graphs P_j, the interoperation of PQ-trees and order graphs, how to determine whether consistent(T_j, P_j) is non-empty, and the resulting CLP algorithm.

Let $G = (V, E)$ be a proper level graph. The order graph $P_k = (V, F)$ for $G = G_k$ is initially the empty digraph on the same vertex set as G. Edges are then added step by step. First, for any vertex pair $u, v \in V_j$ on some level $j \in [k]$ with $u \prec' v$, the edge (u, v) is added to F. Next, according to Lemma 2.1, if $(u, w) \in F$ and $uw \equiv vx$, then (v, x) is added to F. To ensure transitivity, if $(u, v), (v, w) \in F$, then (u, w) is added to F as well. These last two steps are repeated until no more edges can be added to F. The level-j order graph P_j is the order graph for G_j.

Adding edges from the constraints takes time linear in the size of $\prec'$. Adding edges because of Lemma 2.1 is possible by considering all quadratically many edge pairs in E. The other edges can be added by running a transitive closure algorithm, e.g., the Floyd-Warshall algorithm [36], which requires cubic time. These last two steps are repeated at most $\mathcal{O}(n^2)$ times, because F can contain at most n^2 edges. This gives an overall running time of $\mathcal{O}(n^5)$.

The next step is to determine whether a PQ-tree T and an order graph P are consistent, i.e., whether consistent$(T, P) \neq \emptyset$. To this end, we use a recursive procedure similar to the one by Klavík et al. [29, Step 3]. First, it picks an inner node Y of T that has only leaves as children. If $P[Y] := P[\text{yield}(\text{pertinent}(Y))]$ has no suitable topological ordering, T and P are inconsistent. Otherwise, $T \diamondsuit Y$ is generated from T by removing all children of Y, making it a leaf, and $P \circ Y$ is generated from P by identifying the vertices in yield(pertinent(Y)) into a single vertex Y; see Figure 3. Then T and P are consistent if and only if $T \diamondsuit Y$ and $P \circ Y$ are consistent. This is formalized as follows.

LEMMA 3.4. *Let T be a PQ-tree and P an order graph with identical yields. Let Y be an inner node of T so that all its children $y_1, y_2, \ldots, y_t$ are leaves. If Y is a P-node and $P[Y]$ has no topological ordering, or if Y is a Q-node and neither $(y_1, y_2, \ldots, y_t)$ nor $(y_t, y_{t-1}, \ldots, y_1)$ are topological orderings of $P[Y]$, then T and P are inconsistent. Otherwise, T and P are consistent if and only if $T \diamondsuit Y$ and $P \circ Y$ are consistent.*

The running time of the consistency procedure is linear in the size of the order graph P, or $\mathcal{O}(n^2)$ across all order graphs. Note that instead of destroying T by calculating $T \diamondsuit Y$, one can simply mark Y as a leaf. The consistency procedure then applies equivalence transformations on T to make its frontier a topological ordering of P, i.e., reordering T according to P. To prove the correctness of the algorithm, the following relationship between inner nodes of the PQ-tree and order graphs is used.

LEMMA 3.5. *Let G be a single-source planar proper level graph and let T be the corresponding PQ-tree for level j. Further, let Y be a Q-node in T with ordered children $Y_1, Y_2, \ldots, Y_t$ and let*

$$u \in \text{yield}(\text{pertinent}(Y_a)), \quad w \in \text{yield}(\text{pertinent}(Y_b)),$$
$$v \in \text{yield}(\text{pertinent}(Y_c)), \quad x \in \text{yield}(\text{pertinent}(Y_d)),$$

where $1 \leq a < b \leq t$ and $1 \leq c < d \leq t$. If Y is a Q-node, or if Y is a P-node with $c = a$ and $d = b$, then $uw \equiv vx$.

Proof. First, assume that Y is a P-node, as shown in Figure 4 (a). Lemma 3.3 gives that Y corresponds to a cutvertex y in G. Since u and w belong to the yields of pertinent subtrees of distinct children $Y_a \neq Y_b$ of Y, there are paths $\langle u, y_a, y\rangle$, $\langle w, y_b, y\rangle$ in G that have y as the only common vertex. As G is a single-source graph, y_a and y_b belong to the same level and are peaks of the respective paths. Lemma 2.2 gives $uw \equiv y_a y_b$. The same idea yields $vx \equiv y_a y_b$, which gives $uw \equiv vx$. Now assume that Y is a Q-node, as shown in Figure 4 (b). Lemma 4 gives that Y corresponds to a maximal biconnected component H in G. Since G is planar, H is bounded by a circle C, and since G is single-source, C has exactly one peak p. This peak has two children p_1, p_2 on C, regardless of the choice of a, b, c, d. As u and w to the yields of pertinent subtrees of distinct children of Y, there are disjoint paths $\langle u, y_a, p_1\rangle$ and $\langle w, y_b, p_2\rangle$ in G, where y_a and y_b are the first vertices on the paths that lie on C. Lemma 2.2 gives $uw \equiv p_1 p_2$. Applying the same idea yields $vx \equiv p_1 p_2$, and therefore $uw \equiv vx$. □

LEMMA 3.6. *Let G be a single-source proper k-level graph together with constraints $\prec'$, let T_j be the PQ-tree for some level j and let P_j be the order graph for level j in step j. Then it is $(v_1, v_2, \ldots, v_t) \in$ consistent(T_j, P_j) if and only if there*

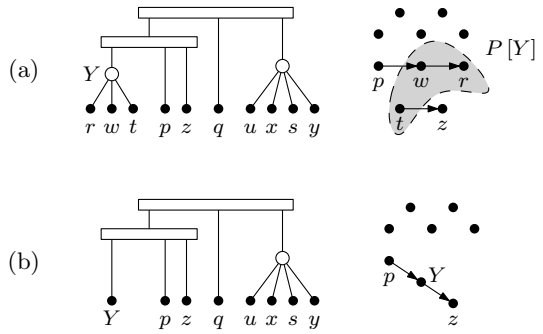

Figure 3: Illustration of Lemma 3.4. Part (a) shows T and P, and (b) shows $T \diamondsuit Y$ and $P \circ Y$.

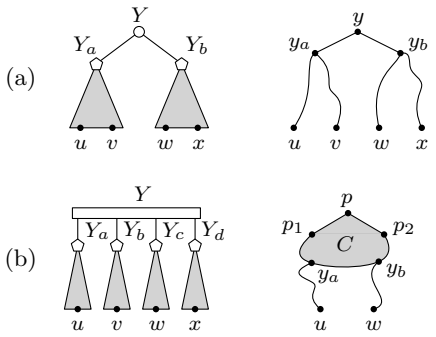

Figure 4: Proof of Lemma 3.5 for P-nodes and Q-nodes in (a) and (b), respectively.

exists a planar drawing $\prec_j$ of G_j that is compatible with $\prec'_j$ and satisfies $v_1 \prec_j v_2 \prec_j \ldots \prec_j v_t$.

Proof. Consider any planar drawing $\prec_j$ of G_j that extends $\prec'_j$ with $v_1 \prec_j v_2 \prec_j \ldots \prec_j v_t$. Since $\prec_j$ is planar, Lemma 3.2 gives $\pi := (v_1, v_2, \ldots, v_t) \in \text{consistent}(T_j)$. Since $\prec_j$ extends $\prec'_j$, and only necessary edges have been added to the order graphs, it is $\pi \in \text{consistent}(P_j)$. Together, this gives $\pi \in \text{consistent}(T_j, P_j)$. Now consider the reverse direction. Since G has a single source, the claim is trivially true for $j = 1$. Assume that the claim is true for $j \in [k-1]$. Lemma 3.2 gives that any order in $\text{consistent}(T_{j+1}, P_{j+1})$ corresponds to a planar drawing of G_{j+1}. Lemma 3.5 states that T_{j+1} can be reordered according to P_{j+1} without causing any conflicts with reordering T_j according to P_j. Hence, the claim is also true for $j+1$. □

3.3 An Efficient PLP Algorithm

The running time of the PLP algorithm from the previous section can be improved to $\mathcal{O}(n + \ell)$, where ℓ is the number of constraints in the partial drawing $\prec'$. To this end, order graphs and PQ-trees are merged into a single new data structure, the constrained PQ-tree, which is based on the PC-tree data structure.

A constrained PQ-tree is a PC-tree T that stores additional edge constraints. An *edge constraint* is a tuple of edges (e, f) of the tree that have the same parent node in the tree with the meaning that e must occur before f w.r.t. the internal rooting. This information is stored in doubly linked constraint lists at both e and f; see Figure 1 for an example (constraints and parent edges are colored). Storing these lists requires one pointer to the head of the list for each edge and two list nodes of constant size for every constraint. Hence, T has a total size of $\mathcal{O}(n + \ell)$.

During the update procedure, a constraint (e, f) may need to be replaced. This is the case when their parent node is split so that the edges corresponding to e and f end up at different parent nodes after the update. In the following we distinguish cases based on whether e and f are terminal, black, or white.

Assume that e is terminal. Then (e, f) needs to be replaced by the equivalent constraint (e', f), where e' is determined as follows. Let Y be the parent node of e and f and let $e = (Y, Z)$. Then e' is the black child edge of Z that is a neighbor of e. This edge can be found in constant time. The same procedure can be used if f is terminal. At this point, no constraints involve terminal edges. Now, a constraint has to be moved if one edge is white, and the other edge is black. Assume that e is white and f is black. Then (e, f) is replaced by the constraint $((Y, x), (Y', x))$, where Y' is the split node of Y. Analogously, if e is black and f is white, (e, f) is replaced by $((Y', x), (Y, x))$. Note that all replacement operations have constant running time. All other constraints do not have to be replaced; see Figure 1 for an illustration of the update procedure for a constrained PQ-tree with three constraints.

Let $T' = \text{update}(T, S)$ and $T'' = \text{update}(T', S')$ with $S \cap S' = \emptyset$. If some edge e of T is black or terminal during the $\text{update}(T, S)$ operation and it exists in T', it is white in T' during the $\text{update}(T', S')$ operation. Only constraints that involve a black or terminal edge are considered for replacement. Hence, any constraint is considered for replacement at most twice, leading to an additional running time in $\mathcal{O}(\ell)$ across all update operations for one level. Walking through the tree to consider all relevant constraints for replacement requires an additional running time in $\mathcal{O}(|S| + p)$ for a single update operation. The original PC-tree update procedure is run as part of this new procedure. With Lemma 2.4, this gives a total running time of $\mathcal{O}(n + k\ell)$ for all update operations of the constrained PQ-tree T over the k levels.

To check whether a constrained PQ-tree has a

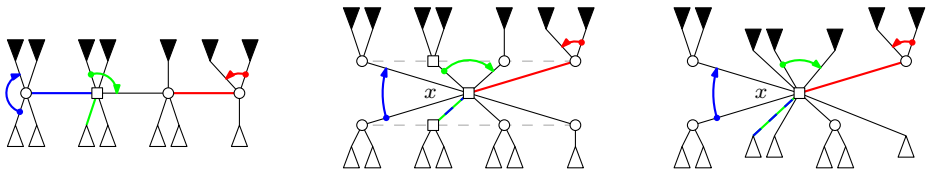

Figure 5: The update procedure for constrained PQ-trees. Constraints are drawn in colors, together with their respective parent edges. The blue constraint involves a white and a black edge, the green constraint involves a black edge and a terminal edge, and the red constraint involves two black edges.

frontier that is consistent with the stored constraints, we use the consistency procedure from Section 3.2. The running time of this procedure for level j is $\mathcal{O}(n_j + \ell)$, where n_j denotes the number of vertices with level j. To solve PLP, run the PLP algorithm from Section 3.1 to compute the constrained PQ-tree T_j for each level j, then add all level-j constraints and run the consistency procedure. The partial drawing can be extended to a complete planar drawing of G if and only if T_j is consistent for each level. We conclude the following.

THEOREM 3.1. *For single-source proper k-level graphs* CLP *can be solved in time $\mathcal{O}(n + k\ell)$, where n is the number of vertices in the graph and ℓ is the size of the constraints. In particular,* PLP *can be solved in $\mathcal{O}(kn)$ time for single source proper k-level graphs.*

It is, of course an interesting question whether the algorithm can be extended to graphs with multiple sources. Any level graph with multiple sources can be transformed into a level graph with one source by adding one vertex and some edges, without changing its planarity. To see this, consider a planar drawing of a graph. Any level-j source can be removed by adding an edge to a suitable vertex on level $j-1$. The difficulty of this procedure is to find out *which* edges to add. If the input graph G has a fixed number r of sources, one can simply try all possible combinations of edges. There are at most n choices per source, leading to a total of n^r choices. For each of them we employ the algorithm from Theorem 3.1.

COROLLARY 3.1. *For proper k-level graphs with r sources,* CLP *can be solved in time $\mathcal{O}(n^r(n + k\ell))$.*

4 Complexity of the General Case

Jünger et al. [25] extend Di Battista and Nardelli's LEVEL PLANARITY algorithm for multi-source graphs. To this end, they run an instance of Di Battista and Nardelli's algorithm for every source of the graph. As soon as the algorithm has progressed to a level j where multiple sources belong to the same connected component in G_j, the corresponding PQ-trees are merged. The order graphs from the previous section cannot simply be reused in the multi-source case, because neither does Lemma 3.5 apply to multi-source graphs, nor is it obvious how constraints should be handled during the merge operation. In fact, it is unlikely that order graphs can be used for multi-source graphs as well, because in this section we show that PLP is $\mathcal{NP}$-complete in the general case by giving a reduction from the $\mathcal{NP}$-hard problem PLANAR MONOTONE 3-SAT [13].

Recall that 3-SAT is the problem of deciding, given a set of variables V and a set of clauses $\mathcal{C}$ where every clause $C \in \mathcal{C}$ contains at most three literals, whether there is a Boolean truth assignment for the variables V so that every clause contains at least one literal that evaluates to "true". A clause is called *positive* if it contains only positive literals, and *negative* if it only contains negative literals. A 3-SAT instance is monotone if it contains only positive and negative clauses. Further limitations are expressed in terms of the *variable-clause graph*. The vertices of this graph are $V \cup \mathcal{C}$ and a vertex v and a clause $\mathcal{C}$ are connected by an edge if and only if v or $\neg v$ occurs in $\mathcal{C}$. The variable-clause graph of PLANAR MONOTONE 3-SAT instances can be drawn in such a way that all variable vertices are drawn as squares on a vertical straight line, the *line of variables*, clauses are drawn as rectangles of variable height and all edges are drawn as horizontal straight lines, and positive clauses are drawn to the right of the line of variables and negative clauses are drawn to the left of the line of variables. The left half of Figure 6 shows a variable-clause graph. Note that every literal in a PLANAR MONOTONE 3-SAT instance can be assumed to occur in at most three clauses.

To transform a PLANAR MONOTONE 3-SAT instance $\mathcal{I}$ into a PLP instance $(G, \prec')$, parts of the variable clause graph are replaced by level graph gadgets together with partial drawings thereof. First, the drawing is altered so that all edges are drawn as y-monotone curves that connect to the bottom of clause rectangles and to the top of variable rectangles, as shown in the right half of Figure 6. Then, every variable is replaced by a *variable gadget*, every clause is replaced by a *clause*

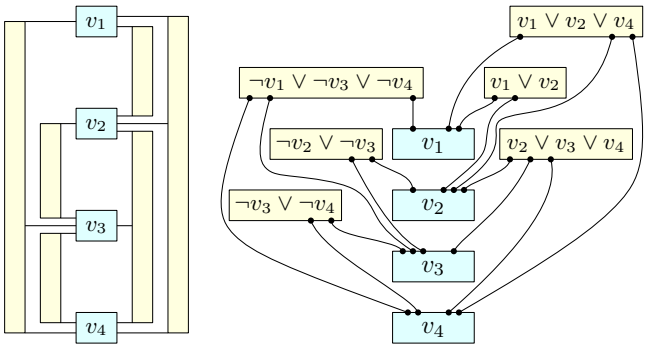

Figure 6: A variable-clause graph on the left, and the modified version with y-monotone curves on the right.

gadget and every edge is replaced by a *pipe gadget*.

Before we describe the construction of the gadgets, we first present two basic building blocks, *sockets* and *plugs*, that play a crucial role in all constructions; see Figure 7. A plug is simply a path in the graph that traverses several layers in a zig-zag fashion, first up, then down and then up again. Plugs are not affected by any constraints. A socket has a fixed drawing and consists of two paths forming the left and the right boundary and two paths that are attached to the upper left corner of the boundary and to the bottom right corner of the boundary, respectively. Their level assignment is such that the endpoints of these latter paths are on the same level, and they are drawn in such a way that their endpoints are drawn between the two boundary paths and that the endpoint of the path starting from the top left corner is to the right of the endpoint of the path starting from the bottom right corner. The crucial property that we exploit throughout our constructions is that in a level-planar drawing at most one plug can be embedded between two boundary paths of a socket.

The pipe gadget consists of two channels that are fixed by the partial drawing, and one conductor that remains to be drawn. Figure 9 shows a pipe gadget, where the channels are drawn in black and the conductor is drawn in blue. Depending on the completion of the partial drawing, the conductor may flow through either the left or the right channel. Each pipe gadget is merged with a clause gadget at the top and with variable gadget at the bottom. This way, information is relayed between clause gadgets and variable gadgets.

The variable gadget consists of six merged pipe ends, separated in the middle by a separator vertex, and three plugs merged at their lower endpoint. This plug structure must lie either completely to the left of the separator, or completely to the right. Depending on this, the variable is configured as "true" or "false". Figure 10 shows a variable gadget configured as "false".

LEMMA 4.1. *If a variable gadget is configured as "false" ("true"), the conductors of all pipes leading to positive (negative) clauses must run through the right channel. The channel choice of the conductors of all pipes leading to negative (positive) clauses is not restricted.*

The clause gadget consists of three merged pipe sources and a singular plug. Figure 8 shows a clause gadget where the first literal is satisfied. This plug must be drawn in one of the pipe sources, thereby forcing the conductor of that pipe to flow through the left channel.

LEMMA 4.2. *For each clause gadget, the conductor of at least one of the three connected pipes must run through the left channel.*

Let $\mathcal{I}$ be an instance of PLANAR MONOTONE 3-SAT and let $(G, \prec')$ be the corresponding PLP instance. Assume that there exists a planar completion of $\prec'$. Consider a positive clause C. Lemma 4.2 states that the conductor of at least one of the connected pipes must run through the left channel. Suppose that the variable V at the other end of this pipe is configured as "false". Then Lemma 4.1 states that the conductors of all pipes leading to positive clauses, including C, must run through the right channel. This is a contradiction, so V must be configured as "true" and C is satisfied. A similar argument can be made when C is a negative clause.

Conversely, suppose that all variable gadgets are configured according to a satisfying assignment. Consider a positive clause C. Since C is satisfied, at least one of its variables V is "true". Let the conductor of the pipe P connecting C and V flow through the left channel. This is possible because according to Lemma 4.1, only the conductors of pipes leading to negative clauses must run through the right channel. A similar argument can be made when C is a negative clause. Any remaining conductors may flow through either channel.

Hence, a planar completion of $\prec'$ exists if and only if there is a satisfying variable assignment for $\mathcal{I}$. With the $\mathcal{NP}$-completeness of PLANAR MONOTONE 3-SAT, this gives the following.

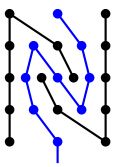

Figure 7: A socket (black) and a plug (blue). Note that embedding two plugs on the same levels between the boundary paths of the same socket necessarily creates a crossing.

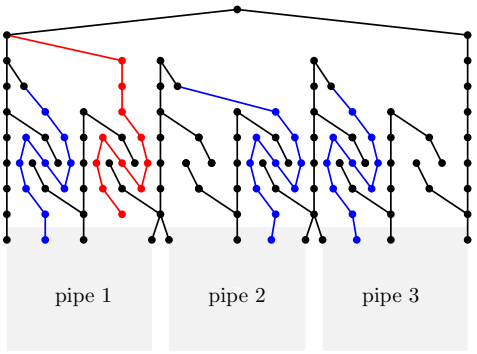

Figure 8: The clause gadget; pipes 1 and 3 transmit a literal value that satisfies the clause.

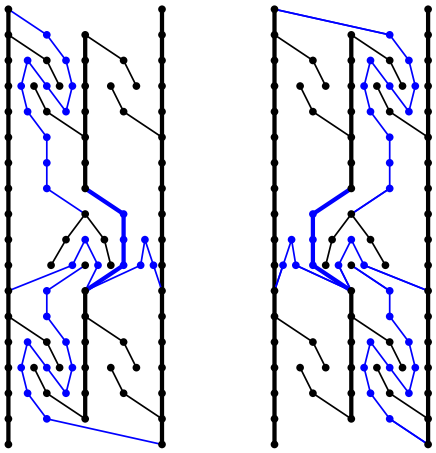

Figure 9: The two states of a pipe gadget. Note the construction in the middle, which serves to connect the inner part separating the two channels to the boundary.

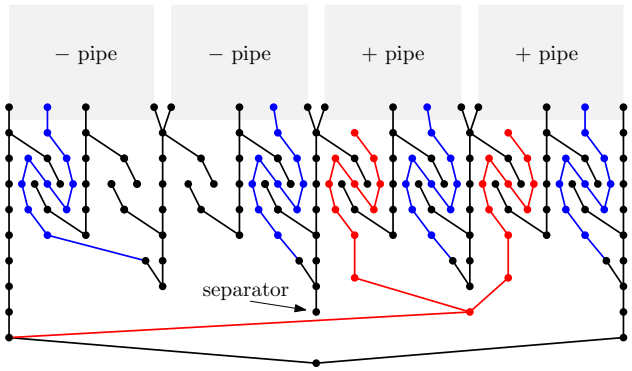

Figure 10: The variable gadget. For space reasons only two pipes are attached on each side. Here, the variable is in state *false*.

THEOREM 4.1. *The problem* PLP *is* $\mathcal{NP}$*-complete even for connected proper level graphs.*

5 Conclusion

In this paper we studied a constrained version of level planarity that allows to specify constraints on the linear order of vertices in the sought drawing. This problem contains as a special case the problem of extending a partial drawing in the level-planar setting, a problem that has recently received considerable interest for many other drawing styles and also other graph representations.

Our strong hardness results leave little room for polynomial-time algorithms for much larger classes of instances than single-source graphs. In view of Corollary 3.1, our most important open question is whether CLP is FPT with respect to the number of sources in the graph.

Another interesting question is whether our techniques can be adapted for related drawing styles, in particular radial drawings, where levels are represented by concentric circles rather than horizontal lines, and upward drawings, where the levels of vertices are not fixed.

Acknowledgments. We thank Giordano Da Lozzo for the idea behind the plug/socket gadget.

References

[1] Patrizio Angelini, Giuseppe Di Battista, Fabrizio Frati, Maurizio Patrignani, and Ignaz Rutter. Testing the simultaneous embeddability of two graphs whose intersection is a biconnected or a connected graph. *J. Discrete Algorithms*, 14:150–172, 2012.

[2] Patrizio Angelini, Giordano Da Lozzo, Giuseppe Di Battista, Fabrizio Frati, and Vincenzo Roselli. The importance of being proper: (In clustered-level planarity and T-level planarity). *Theoretical Computer Science*, 571:1–9, 2015.

[3] Patrizio Angelini, Giuseppe Di Battista, Fabrizio Frati, Vít Jelínek, Jan Kratochvíl, Maurizio Patrignani, and Ignaz Rutter. Testing Planarity of Partially Embedded Graphs. In *Proceedings of the 21st Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA '10, pages 202–221. SIAM, 2010.

[4] Patrizio Angelini, Giordano Da Lozzo, Giuseppe Di Battista, Fabrizio Frati, Maurizio Patrignani, and Ignaz Rutter. Testing cyclic level and simultaneous level planarity. *CoRR*, abs/1510.08274, 2015.

[5] Christian Bachmaier, Franz-Josef Brandenburg, and Michael Forster. Radial level planarity testing and embedding in linear time. *J. Graph Algorithms Appl.*, 9(1):53–97, 2005.

[6] Thomas Bläsius, Annette Karrer, and Ignaz Rutter. Simultaneous embedding: Edge orderings, relative posi-

tions, cutvertices. In Stephen K. Wismath and Alexander Wolff, editors, *Proceedings of the 21st International Symposium on Graph Drawing (GD'13)*, volume 8242 of *LNCS*, pages 220–231. Springer, 2013.

[7] Thomas Bläsius, Stephen G. Kobourov, and Ignaz Rutter. Simultaneous embedding of planar graphs. In Roberto Tamassia, editor, *Handbook of Graph Drawing and Visualization*. CRC Press, 2013.

[8] Thomas Bläsius and Ignaz Rutter. Disconnectivity and relative positions in simultaneous embeddings. In Walter Didimo and Maurizio Patrignani, editors, *Proceedings of the 20th International Symposium on Graph Drawing (GD'12)*, volume 7704 of *Lecture Notes in Computer Science*, pages 31–42. Springer, 2013.

[9] Thomas Bläsius and Ignaz Rutter. Simultaneous PQ-Ordering with Applications to Constrained Embedding Problems. *ACM Transactions on Algorithms*, 12(2):16:1–16:46, December 2015.

[10] Kellogg S. Booth and George S. Lueker. Testing for the Consecutive Ones Property, Interval Graphs, and Graph Planarity Using *PQ*-Tree Algorithms. *Journal of Computer and System Sciences*, 13(3):335–379, 1976.

[11] Franz J. Brandenburg. Upward planar drawings on the standing and the rolling cylinders. *Compuational Geometry*, 47(1):25–41, 2014.

[12] Steven Chaplick, Radoslav Fulek, and Pavel Klavík. Extending Partial Representations of Circle Graphs. In Stephen Wismath and Alexander Wolff, editors, *Graph Drawing: 21st International Symposium, GD 2013, Bordeaux, France, 2013, Revised Selected Papers*, pages 131–142. Springer International Publishing, 2013.

[13] Mark de Berg and Amirali Khosravi. Optimal Binary Space Partitions in the Plane. In *Computing and Combinatorics, 16th Annual International Conference, COCOON 2010, Nha Trang, Vietnam, July 19-21, 2010. Proceedings*, pages 216–225, 2010.

[14] Giuseppe Di Battista and Enrico Nardelli. Hierarchies and Planarity Theory. *IEEE Transactions on Systems, Man, and Cybernetics*, 18(6):1035–1046, December 1988.

[15] Alejandro Estrella-Balderrama, Elisabeth Gassner, Michael Jünger, Merijam Percan, Marcus Schaefer, and Michael Schulz. Simultaneous geometric graph embeddings. In Seok-Hee Hong, Takao Nishizeki, and Wu Quan, editors, *Proceedings of the 15th International Symposium on Graph Drawing (GD'07)*, volume 4875 of *Lecture Notes in Computer Science*, pages 280–290. Springer, 2007.

[16] Radoslav Fulek, Michael J. Pelsmajer, Marcus Schaefer, and Daniel Štefankovič. *Thirty Essays on Geometric Graph Theory*, chapter Hanani-Tutte, Monotone Drawings, and Level-Planarity. Springer, 2013.

[17] Luca Grilli, Seok-Hee Hong, jan Kratochvíl, and Ignaz Rutter. Drawing simultaneously embedded graphs with few bends. In *Proceedings of the 22nd International Symposium on Graph Drawing (GD'14)*, volume 8871 of *Lecture Notes in Computer Science*, pages 40–51. Springer, 2014.

[18] Martin Harrigan and Patrick Healy. Practical Level Planarity Testing and Layout with Embedding Constraints. In Seok-Hee Hong, Takao Nishizeki, and Wu Quan, editors, *Graph Drawing: 15th International Symposium, GD 2007, Sydney, Australia, 2007 Proceedings*, pages 62–68. Springer Berlin Heidelberg, 2008.

[19] Patrick Healy and Nikola S. Nikolov. *Handbook of Graph Drawing and Visualization*, chapter Hierarchical Drawing Algorithms. CRC Press, 2013.

[20] Wen-Lian Hsu and Ross M. McConnell. *PC* Trees and Circular-Ones Arrangements. *Theoretical Computer Science*, 296(1):99–116, 2003. Computing and Combinatorics.

[21] Krishnam R. Jampani and Anna Lubiw. Simultaneous interval graphs. In *Algorithms and Computation - 21st International Symposium, ISAAC 2010, Jeju Island, Korea, December 15-17, 2010, Proceedings, Part I*, pages 206–217, 2010.

[22] Krishnam R. Jampani and Anna Lubiw. The simultaneous representation problem for chordal, comparability and permutation graphs. *J. Graph Algorithms Appl.*, 16(2):283–315, 2012.

[23] Vít Jelínek, Jan Kratochvíl, and Ignaz Rutter. A kuratowski-type theorem for planarity of partially embedded graphs. *Computational Geometry Theory & Applications*, 46(4):466–492, 2013.

[24] Michael Jünger and Sebastian Leipert. Level Planar Embedding in Linear Time. In Jan Kratochvíyl, editor, *Graph Drawing: 7th International Symposium, GD'99, Štiřín Castle, Czech Republic, 1999 Proceedings*, pages 72–81. Springer Berlin Heidelberg, 1999.

[25] Michael Jünger, Sebastian Leipert, and Petra Mutzel. Level Planarity Testing in Linear Time. In Sue H. Whitesides, editor, *Graph Drawing: 6th International Symposium, GD '98, Montréal, Canada, 1998 Proceedings*, pages 224–237. Springer Berlin Heidelberg, 1998.

[26] Pavel Klavík, Jan Kratochvíl, Tomasz Krawczyk, and Bartosz Walczak. Extending Partial Representations of Function Graphs and Permutation Graphs. In Leah Epstein and Paolo Ferragina, editors, *Algorithms – ESA 2012: 20th Annual European Symposium, Ljubljana, Slovenia, September 10-12, 2012. Proceedings*, pages 671–682. Springer Berlin Heidelberg, 2012.

[27] Pavel Klavík, Jan Kratochvíl, Yota Otachi, Ignaz Rutter, Toshiki Saitoh, Maria Saumell, and Tomáš Vyskočil. Extending Partial Representations of Proper and Unit Interval Graphs. In R. Ravi and Inge Li Gørtz, editors, *Algorithm Theory – SWAT 2014: 14th Scandinavian Symposium and Workshops, Copenhagen, Denmark, July 2-4, 2014. Proceedings*, pages 253–264. Springer International Publishing, 2014.

[28] Pavel Klavík, Jan Kratochvíl, Yota Otachi, and Toshiki Saitoh. Extending Partial Representations of Subclasses of Chordal Graphs. *Theoretical Computer Science*, 576:85–101, April 2015.

[29] Pavel Klavík, Jan Kratochvíl, and Tomáš Vyskočil.

Extending Partial Representations of Interval Graphs. In Mitsunori Ogihara and Jun Tarui, editors, *Theory and Applications of Models of Computation*, volume 6648 of *Lecture Notes in Computer Science*, pages 276–285. Springer Berlin Heidelberg, 2011.

[30] Tamara Mchedlidze, Martin Nöllenburg, and Ignaz Rutter. Extending convex partial drawings of graphs. *Algorithmica*, 2015.

[31] Maurizio Patrignani. On Extending a Partial Straight-Line Drawing. In Patrick Healy and Nikola S. Nikolov, editors, *Graph Drawing: 13th International Symposium, GD 2005, Limerick, Ireland, 2005 Proceedings*, pages 380–385. Springer Berlin Heidelberg, 2006.

[32] Bert Randerath, Ewald Speckenmeyer, Endre Boros, Peter Hammer, Alex Kogan, Kazuhisa Makino, Bruno Simeone, and Ondrej Cepek. A Satisfiability Formulation of Problems on Level Graphs. *Electronic Notes in Discrete Mathematics*, 9:269–277, 2001.

[33] Marcus Schaefer. Toward a theory of planarity: Hanani-tutte and planarity variants. *Journal of Graph Algorithms and Applications*, 17(4):367–440, 2013.

[34] Wei-Kuan Shih and Wen-Lian Hsu. A New Planarity Test. *Theoretical Computer Science*, 223(1–2):179–191, 1999.

[35] Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiko Toda. Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2):109–125, 1981.

[36] Stephen Warshall. A Theorem on Boolean Matrices. *Journal of the ACM*, 9(1):11–12, January 1962.